



ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ  
Εθνικόν και Καποδιστριακόν  
Πανεπιστήμιον Αθηνών  
— ΙΔΡΥΘΕΝ ΤΟ 1837 —

2026

*ΠΜΣ στην Διοίκηση, Αναλυτική και Πληροφοριακά Συστήματα*

# Machine Learning Algorithms with R

REGRESSION AND CLASSIFICATION TREES

ELENI KALTSOUNI A.M: 7341122500013

## Περιεχόμενα

|                                                                  |    |
|------------------------------------------------------------------|----|
| 1. Άσκηση 1 – Ανάλυση του Συνόλου Δεδομένων Boston .....         | 2  |
| 2. Άσκηση 2 – Δέντρα Ταξινόμησης με το OJ Dataset .....          | 7  |
| 3. Άσκηση 3 – Δέντρα Παλινδρόμησης με το Car seats Dataset ..... | 14 |

## 1. Άσκηση 1 – Ανάλυση του Συνόλου Δεδομένων Boston

- i. Πόσες γραμμές και πόσες στήλες περιέχονται στο σύνολο δεδομένων; Τι αντιπροσωπεύουν οι γραμμές και τι οι στήλες;

Ξεκινώντας τη στατιστική ανάλυση, αρχικά φορτώνουμε την βιβλιοθήκη MASS και το σύνολο των δεδομένων Boston. Με την εκτέλεση της γραμμής `dim(Boston)`, στην R, βλέπουμε ως αποτέλεσμα ότι το σύνολο των δεδομένων έχει 506 γραμμές ή παρατηρήσεις και 14 στήλες ή μεταβλητές.

Οι γραμμές αντιπροσωπεύουν τα διαφορετικά προάστια και τις γειτονιές της Βοστώνης, ενώ οι στήλες αντιπροσωπεύουν τα οικονομικά, δημογραφικά και περιβαλλοντικά χαρακτηριστικά των προαστίων αυτών (π.χ. δείκτης εγκληματικότητας και αναλογία μαθητών – δασκάλων)

```
> dim(Boston)
[1] 506 14
```

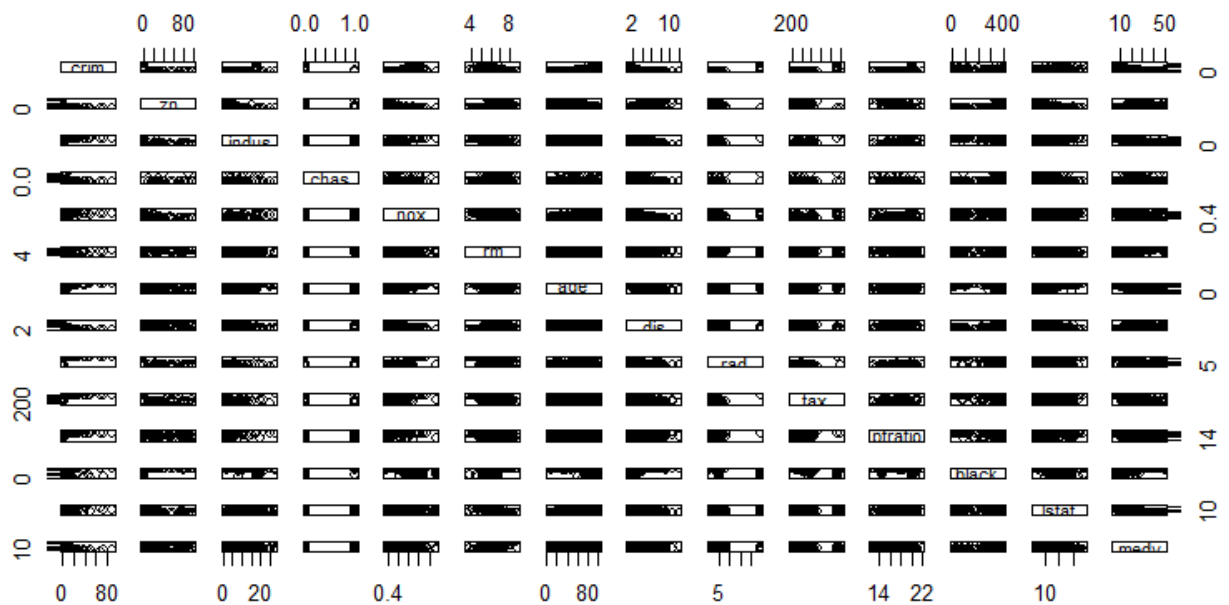
Χρησιμοποιώντας τη συνάρτηση `str(Boston)`, λαμβάνουμε ως αποτέλεσμα τον τύπο των μεταβλητών και παρατηρούμε ότι όλες είναι αριθμητικού τύπου (numeric, integer), γεγονός που διευκολύνει την ποσοτική τους ανάλυση.

- ii. Scatter plots όλων των μεταβλητών ανά δύο και scatter plot δύο οποιονδήποτε μεταβλητών (π.χ. crim, age)

Με την συνάρτηση `pairs(Boston)`, δημιουργήθηκε ένα πίνακας διαγραμμάτων διασποράς για όλες τις μεταβλητές ανά δύο και αποτελεί μέρος της Εξερευνητικής Ανάλυσης Δεδομένων, καθώς απεικονίζει γραμμικές και μη-γραμμικές σχέσεις μεταξύ των μεταβλητών του συνόλου δεδομένων.

Παρατηρώντας την πρώτη γραμμή που αντιστοιχεί στο κατά κεφαλήν ποσοστό της εγκληματικότητας (**crim**), διαπιστώνουμε ότι η εγκληματικότητα δεν συνδέεται ομοιόμορφα με όλες τις μεταβλητές. Αντίθετα, εμφανίζει έντονη συσχέτιση με μεταβλητές όπως:

- (**age**) – Παλαιότητα: Παρατηρείται έξαρση εγκληματικότητας στις παλαιότερες περιοχές
- (**tax**) - Φόροι: Συγκέντρωση υψηλών ποσοστών εγκληματικότητας σε περιοχές με μεγάλη φορολογία.
- (**dis**) - Απόσταση από κέντρα εργασίας: Η εγκληματικότητα μειώνεται όσο απομακρυνόμαστε από εργασιακά κέντρα.
- (**medv**) - Αξία κατοικιών: Αυξημένη συγκέντρωση εγκληματικότητας αποκλειστικά στις περιοχές με χαμηλή αξία κατοικίας.

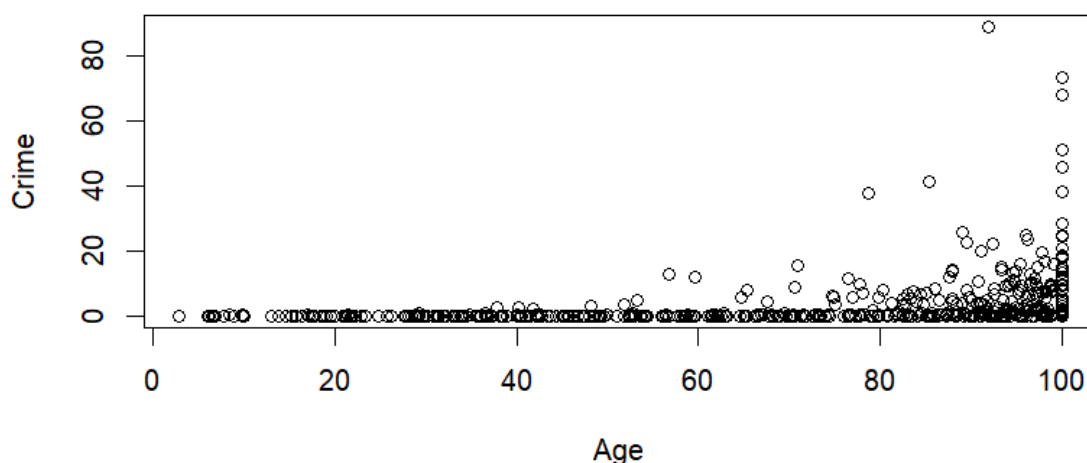


Το διάγραμμα διασποράς Age and Crime μας δείχνει μια έντονα μη-γραμμική σχέση μεταξύ της παλαιότητας των κτιρίων μιας περιοχής και του δείκτη εγκληματικότητας.

Διαπιστώνουμε ότι για τα προάστια με χαμηλή ή μέση ηλικία, δηλαδή έως το 80%, ο δείκτης εγκληματικότητας είναι ελεγχόμενος και αρκετά χαμηλός έως και μηδενικός. Η απότομη αύξηση της εγκληματικότητας γίνεται όταν η παλαιότητα της περιοχής αγγίζει το 100%. Σε αυτό το σημείο, εμφανίζεται μια κατακόρυφη διασπορά των τιμών, με την εγκληματικότητα να αποκτά ακραίες τιμές, δηλαδή δημιουργούνται outliers που ξεπερνούν το 40%-80%.

Συμπεραίνουμε λοιπόν, ότι η εγκληματικότητα στη Βοστώνη δεν είναι απλά ένα γενικευμένο φαινόμενο, αλλά είναι συγκεντρωμένο αποκλειστικά στις παλαιότερες και μάλλον υποβαθμισμένες περιοχές της, όπου οι έλλειψη οικιστικών επενδύσεων συμβάλει στην περαιτέρω αύξησή της.

### Age and Crime



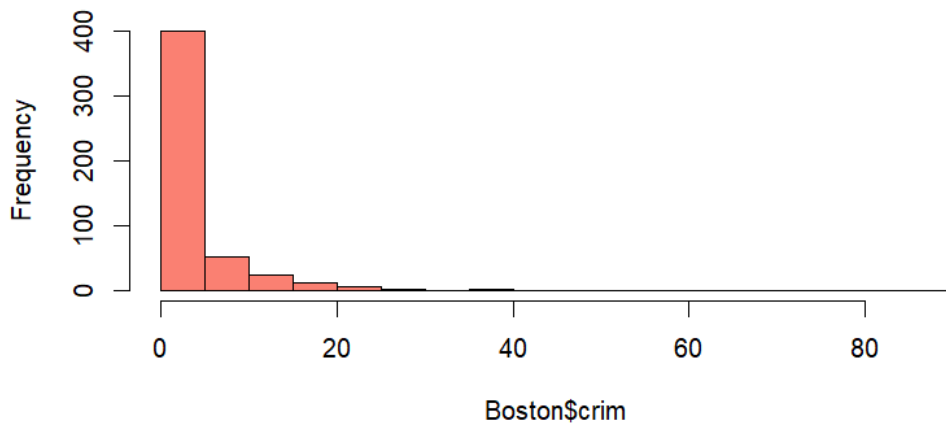
### iii. Ιστογράμματα και Έλεγχος Ακραίων Τιμών

Με τη χρήση της συνάρτησης [hist\(\)](#), κατασκευάσαμε ιστογράμματα 25 κλάσεων για την ανάλυση των κατανομών των μεταβλητών **tax**, **crim** και **ptratio**. Από αυτή τη γραφική απεικόνιση προκύπτουν τα εξής συμπεράσματα:

- **(crim)- Ποσοστά Εγκληματικότητας**

Η κατανομή παρουσιάζει θετική ασυμμετρία (right-skewed). Στην πλειονότητα τα προάστια έχουν ποσοστά εγκληματικότητας κοντά στο μηδέν. Όμως, υπάρχει μια μικρή ομάδα προαστίων που αποτελούν ακραίες τιμές με πολύ υψηλά ποσοστά εγκληματικότητας, τιμές μεγαλύτερες από το 20% που ξεπερνούν ακόμα και το 80%. Αυτό τα καθιστά έδρες έντονης παραβατικότητας.

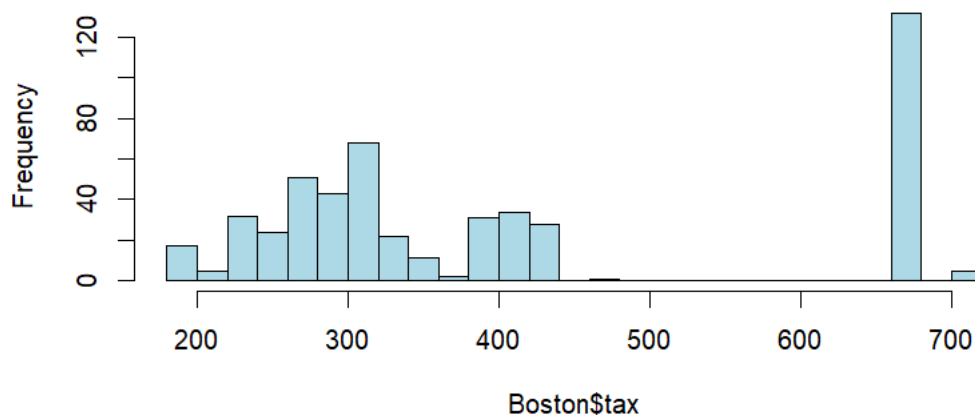
**Crime Histogram**



- **(tax)- Φορολογικοί Δείκτες**

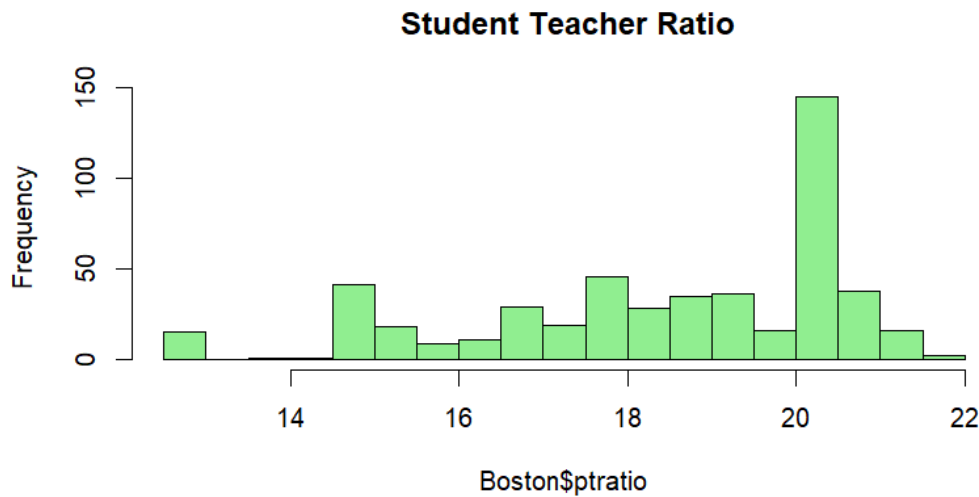
Η κατανομή είναι διτροπική (bimodal). Παρατηρούμε ότι υπάρχει μια μεγάλη συγκέντρωση προαστίων με χαμηλούς έως μεσαίους φόρους (200-400), αλλά και μια απομακρυσμένη, ξεχωριστή κορυφή με πολύ υψηλές τιμές (600-700). Αυτό σημαίνει ότι υπάρχει μια συγκεκριμένη κατηγορία προαστίων που έχει πάρα πολύ υψηλή φορολογία.

**Tax Histogram**



- **(ptratio) – Αναλογία Μαθητών-Δασκάλων**

Η κατανομή εμφανίζει αριστερή ασυμμετρία (left-skewed). Η υψηλότερη συχνότητα-κορυφή εμφανίζεται στην κλάση 20, γεγονός που μας δείχνει ότι ένα μεγάλο μέρος των προαστίων της Βοστώνης έχει υψηλή αναλογία μαθητών ανά δάσκαλο. Αυτό μπορεί να σημαίνει ότι υπάρχει πιθανή υποστελέχωση στα σχολεία αυτών των περιοχών.



#### iv. Προάστια που συνορεύουν με τον ποταμό Charles

Η μεταβλητή **chas** αποτελεί μια δυαδική μεταβλητή (dummy variable), η οποία λαμβάνει την τιμή 1 αν το προάστιο συνορεύει με τον ποταμό Charles, και την τιμή 0 αν δεν συνορεύει. Χρησιμοποιώντας την εντολή που ακολουθεί, προκύπτει ότι τα προάστια που συνορεύουν με τον ποταμό Charles είναι 35.

```
> sum(Boston$chas == 1)
[1] 35
```

Από στατιστικής πλευράς, οι περιοχές που συνορεύουν εν τέλει με τον ποταμό, αποτελούν μια σαφή μειονότητα καθώς είναι μόνο 35 από τα 506 προάστια.

#### v. Διάμεση Αναλογία Μαθητών-Δασκάλων μεταξύ των προαστίων της Βοστώνης.

Η αναλογία μαθητών ανά δάσκαλο (**ptratio**) αποτελεί έναν σημαντικό δείκτη για την αξιολόγηση των εκπαιδευτικών υποδομών της περιοχής. Εκτελώντας την εντολή που ακολουθεί, προκύπτει ότι η διάμεση αναλογία για τα προάστια της Βοστώνης είναι 19,05.

```
> median(Boston$ptratio)
[1] 19.05
```

Αυτό το αποτέλεσμα σημαίνει ότι στο 50% των προαστίων, η αναλογία είναι μεγαλύτερη ή ίση με 19,05 μαθητές ανά δάσκαλο. Η τιμή αυτή επιβεβαιώνει την αριστερή ασυμμετρία που

παρατηρήσαμε στο πάνω ιστόγραμμα, δείχνοντας ότι τα περισσότερα σχολεία στη Βοστώνη λειτουργούν με σχετικά υψηλό βαθμό μαθητών ανά τάξη και έτσι ξεπερνούν το όριο μαθητών ανά δάσκαλο.

#### vi. Προάστια με τη Χαμηλότερη Διάμεση Αξία Κατοικιών.

Για να βρούμε τα προάστια με τη χαμηλότερη διάμεση αξία κατοικιών, εξετάσαμε την ελάχιστη τιμή της μεταβλητής **medv**. Η ελάχιστη τιμή αυτή ισούται με 5 δηλαδή 5.000\$.

Εκτελώντας τον κώδικα που παρουσιάζεται πιο κάτω, βλέπουμε ότι υπάρχουν δύο μόνο προάστια που καταγράφουν αυτή τη χαμηλή τιμή και συγκεκριμένα οι γραμμές 399 και 406 του dataframe.

```
> print(suburbs_min_medv)
      crim zn indus chas   nox    rm age   dis rad tax ptratio  black lstat medv
399 38.3518  0  18.1    0 0.693 5.453 100 1.4896 24 666    20.2 396.90 30.59    5
406 67.9208  0  18.1    0 0.693 5.683 100 1.4254 24 666    20.2 384.97 22.98    5
```

Για να κατανοήσουμε το προφίλ αυτών των περιοχών, συγκρίνουμε τις τιμές των υπόλοιπων μεταβλητών τους με τους μέσους όρους των μεταβλητών της Βοστώνης χρησιμοποιώντας το [summary\(Boston\)](#).

- **crim – Εγκληματικότητα:** Τα δύο προάστια εμφανίζουν υψηλά ποσοστά εγκληματικότητας 38,35% και 67,92%, ενώ ο μέσος όρος του πληθυσμού είναι μόλις 3,61%.
- **Age - Παλαιότητα:** Και στα δύο προάστια η παλαιότητα αγγίζει το 100%, ενώ ο μέσος όρος της πόλης είναι στο 68,57%.
- **Tax - Φορολογία:** Υπόκεινται και τα δύο σε επιβαρυμένη φορολογία, με τον συντελεστή να είναι στο 666, ο οποίος είναι πολύ υψηλότερος από τον μέσο όρο του πληθυσμού στο 408,2.
- **Istat - Κοινωνικό-Οικονομικό Επίπεδο:** Το ποσοστό του πληθυσμού χαμηλής οικονομικής – κοινωνικής τάξης είναι στο 30,59% και 22,98% ενώ του μέσου πληθυσμού της Βοστώνης είναι μόλις 12,65%.

Συμπεραίνουμε λοιπόν, πως τα προάστια με τη μικρότερη αξία κατοικιών αποτελούν τις πιο υποβαθμισμένες, παλαιότερες και κοινωνικά περιθωριοποιημένες περιοχές της πόλης. Χαρακτηρίζονται από ακραία εγκληματικότητα, αυξημένη φορολογία και υποβαθμισμένες υποδομές.

#### vii. Προάστια με περισσότερα από 8 Δωμάτια ανά Κατοικία.

Εφαρμόζοντας το φίλτρο [Boston\\$rm > 8](#) στο σύνολο δεδομένων, βλέπουμε πως υπάρχουν 13 προάστια στα οποία οι κατοικίες έχουν κατά μέσο όρο πάνω από 8 δωμάτια ανά κατοικία.

```
> large_rooms <- Boston[Boston$rm > 8, ]  
> nrow(large_rooms)  
[1] 13
```

Συγκρίνοντας το `summary()` αυτού του υποσυνόλου με τα στατιστικά του γενικού πληθυσμού, αναδεικνύονται τα ιδιαίτερα χαρακτηριστικά αυτών των περιοχών:

- **medv - Διάμεση Αξία Κατοικιών:** Η διάμεση αξία των σπιτιών σε αυτά τα 13 προάστια, εκτινάσσεται στα 44,20, τιμή διπλάσια από τη διάμεσο του γενικού πληθυσμού που είναι στα 21,20.
- **crim- Εγκληματικότητα:** Ο ρυθμός της εγκληματικότητας είναι εξαιρετικά χαμηλός, με μέσο όρο 0,7188 %, τι στιγμή που στον γενικό πληθυσμό ο μέσος όρος είναι 3,61%. Αυτό δείχνει ότι αποτελούν εξαιρετικά ασφαλείς γειτονιές.
- **lstat- Κοινωνικό-Οικονομικό Επίπεδο:** Το ποσοστό του πληθυσμού χαμηλής οικονομικής-κοινωνικής τάξης είναι εξαιρετικά χαμηλό, με μέσο όρο 4,31% έναντι του 12,65% που είναι ο μέσος όρος της πόλης.
- **prratio- Αναλογία μαθητών – δασκάλων:** Τα σχολεία αυτών των περιοχών παρουσιάζουν καλύτερη στελέχωση, με μέση αναλογία 16,36 μαθητές ανά δάσκαλο που είναι χαμηλότερη από τον γενικό μέσο όρο της πόλης 18,46.

Φτάνουμε στο συμπέρασμα ότι τα προάστια με μέσο όρο άνω των 8 δωματίων αντιπροσωπεύουν τις πιο εύπορες, ακριβές και κοινωνικά αναβαθμισμένες περιοχές της Βοστώνης. Πρόκειται για περιοχές με περιορισμένη εγκληματικότητα, υψηλή αγοραστική αξία ακινήτων και καλές εκπαιδευτικές υποδομές.

## 2. Άσκηση 2 – Δέντρα Ταξινόμησης με το OJ Dataset

### a) Train / Test Split

Για τη σωστή αξιολόγηση της ικανότητας του μοντέλου, το αρχικό σύνολο δεδομένων OJ που περιέχει 1070 παρατηρήσεις και 18 στήλες, χωρίστηκε τυχαία με τη χρήση της συνάρτησης [`sample\(\)`](#).

**Σύνολο Εκπαίδευσης (Training Set):** Περιλαμβάνει ένα τυχαίο δείγμα με 800 παρατηρήσεις.

**Σύνολο Ελέγχου (Test Set):** Περιλαμβάνει τις υπόλοιπες 270 παρατηρήσεις που απέμειναν, οι οποίες θα χρησιμοποιηθούν αποκλειστικά για την τελική αξιολόγηση.

Η γεννήτρια τυχαίων αριθμών σταθεροποιήθηκε με την εντολή [`set.seed\(123\)`](#) για να διασφαλίσουμε την αναπαραγωγικότητα των αποτελεσμάτων.



Είναι σημαντικό να αναφέρουμε ότι αν αλλάξουμε το [set.seed\(\)](#) και βάλουμε κάποιον άλλον αριθμό τα αποτελέσματα που θα πάρουμε θα είναι διαφορετικά.

## b) Model Fitting and Summary

Προσαρμόσαμε ένα δέντρο ταξινόμησης με εξαρτημένη μεταβλητή την **Purchase** και ερμηνευτικές μεταβλητές όλες τις υπόλοιπες διαθέσιμες πληροφορίες του dataset. Εκτελώντας τη συνάρτηση `summary()` προκύπτουν τα ακόλουθα βασικά στατιστικά στοιχεία για το αρχικό δέντρο:

- **Variables Used:** Το μοντέλο χρησιμοποίησε τις πιο σημαντικές μεταβλητές για τους εσωτερικούς διαχωρισμούς, με τις κυριότερες να είναι η **loyalCH** (πόσο πιστός είναι ο πελάτης στο brand) και **PriceDiff** (διαφορά τιμής).
- **Terminal Nodes:** Το πλήρες δέντρο αναπτύχθηκε σε 8 τερματικούς κόμβους.
- **Training Error Rate:** Το σφάλμα ταξινόμησης στα δεδομένα εκπαίδευσης διαμορφώνεται στο 16,5%. Αυτό σημαίνει ότι το δέντρο καταφέρνει να προβλέψει σωστά τη συμπεριφορά του 83,5% των πελατών κατά την φάση της εκπαίδευσης.

```
> summary(oj_tree)

Classification tree:
tree(formula = Purchase ~ ., data = oj_train)
Variables actually used in tree construction:
[1] "LoyalCH" "PriceDiff"
Number of terminal nodes: 8
Residual mean deviance: 0.7625 = 603.9 / 792
Misclassification error rate: 0.165 = 132 / 800
```

## c) Text Output of the Tree

Εκτυπώνοντας το αντικείμενο του δέντρου με την συνάρτηση [print\(oj\\_tree\)](#), λαμβάνουμε μια λεπτομερή αρχιτεκτονική του μοντέλου. Επιλέγοντας έναν συγκεκριμένο τερματικό κόμβο (τους ξεχωρίζουμε γιατί τελειώνουν με \*), μπορούμε να διερμηνεύσουμε τη διαδρομή της απόφασης:

```
8) LoyalCH < 0.0356415 56 10.03 MM ( 0.01786 0.98214 ) *
```

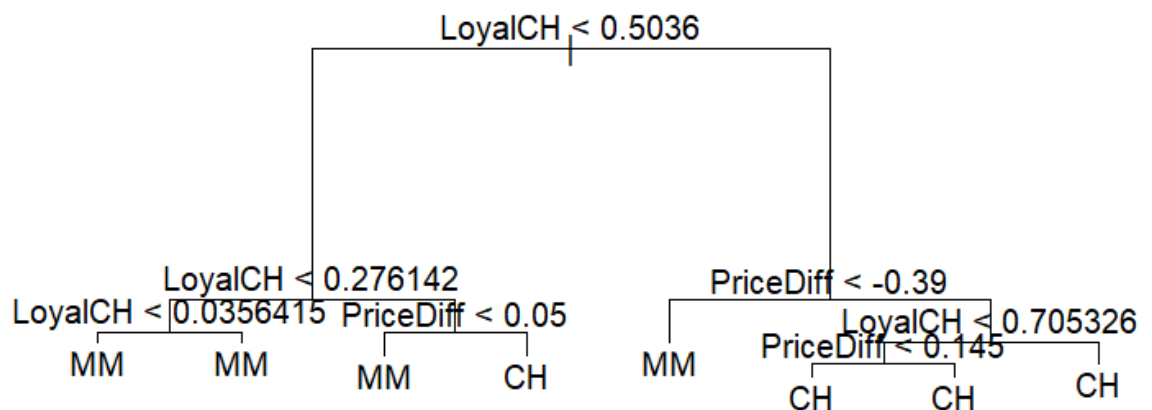
**Ερμηνεία του συγκεκριμένου κόμβου:**

- **Διαδρομή/ Κριτήριο:** Ο συγκεκριμένος κόμβος περιλαμβάνει τους πελάτες που είναι λιγότερο πιστοί στο brand CH. ( $\text{LoyalCH} < 0.0356415$ )
- **Μέγεθος Δείγματος:** Στον συγκεκριμένο κόμβο υπάρχουν 56 πελάτες από το σύνολο εκπαίδευσης.

- **Πρόβλεψη και Πιθανότητες:** Το μοντέλο προβλέπει με υψηλή βεβαιότητα ότι οι επιλογή αυτών των πελατών θα είναι το MM. Συγκεκριμένα η πιθανότητα αγοράς του MM είναι 98,21%, ενώ για το CH είναι μόλις 1,78%.
- **Deviance (Καθαρότητα):** Η τιμή της deviance είναι πολύ χαμηλή 10,03, γεγονός που μας δείχνει ότι ο διαχωρισμός είναι επιτυχημένος και ο κόμβος είναι σχεδόν απόλυτα καθαρός.

#### d) Create the plot and interpret the results.

Απεικονίζουμε τη δομή του δέντρου γραφικά χρησιμοποιώντας τις συναρτήσεις `plot()` και `text()`.



Ο πρώτος και σημαντικότερος διαχωρισμός στην κορυφή του δέντρου γίνεται με βάση τη μεταβλητή **loyalCH**. Αυτό αποδεικνύει ότι το πόσο πιστός είναι ένας πελάτης στο CH είναι ο ισχυρότερος παράγοντας για την τελική αγορά. Πελάτες με υψηλή τιμή **loyalCH** πηγαίνουν προς τη δεξιά πλευρά του δέντρου, όπου υπάρχει ισχυρή πρόβλεψη CH, ενώ οι πελάτες που είναι λιγότερο πιστοί πάνε στην αριστερή πλευρά όπου υπάρχει μεγαλύτερη πρόβλεψη για MM.

Στα χαμηλότερα επίπεδα του δέντρου εξετάζονται δευτερεύοντες παράγοντες όπως είναι η διαφορά στην τιμή. Οι τιμές και οι προσφορές αρχίζουν να επηρεάζουν τους καταναλωτές όταν η μάρκα δεν του είναι αδιάφορη ή όταν η διαφορά της τιμής είναι μεγάλη. Για παράδειγμα, ακόμα και οι πιστοί πελάτες του CH θα στραφούν στο MM, εάν η τιμή του CH είναι υπερβολικά υψηλή. ( `PriceDiff < 0.39`)

#### e) Predictions and Confusion Matrix.

Για να αξιολογήσουμε την ικανότητα του μοντέλου να κάνει προβλέψεις σε νέα δεδομένα, εφαρμόσαμε το `oj_tree` στο Test Set με τις 270 παρατηρήσεις.

Το Confusion Matrix που προκύπτει από την διασταύρωση των πραγματικών τιμών **Purchase** και των προβλέψεων **tree\_pred\_test** μας βοηθά στο να αποκτήσουμε τα τελικά ποσοστά σφάλματος. Καταλαβαίνουμε λοιπόν ότι το μοντέλο ταξινομήσε σωστά 150 αγορές CH και 70 αγορές MM, ενώ απέτυχε σε 50 περιπτώσεις, δηλαδή 34 πελάτες προβλέφθηκαν λανθασμένα σαν CH και 16 ως MM.

```
> table(tree_pred_test, oj_test$Purchase)

tree_pred_test  CH  MM
              CH 150  34
              MM  16  70
```

#### Σύγκριση ποσοστών σφαλμάτων:

- **Training Error Rate:** 16,50%
- **Test Error Rate:** 18,52%

```
> cat("Test Error Rate:", test_error, "\n")
Test Error Rate: 0.1851852
```

```
> cat("Train Error Rate:", train_error, "\n")
Train Error Rate: 0.165
```

Παρατηρούμε ότι το Test Error είναι υψηλότερο από το Train Error. Το αρχικό δέντρο αποθηκεύει το θόρυβο των δεδομένων εκπαίδευσης, αυτό είναι το φαινόμενο του overfitting και μειώνει την γενικευμένη ισχύ του μοντέλου στο Test Set. Έτσι καταλαβαίνουμε ότι είναι απαραίτητο να εφαρμόσουμε Cross-Validation και pruning στο μοντέλο, έτσι ώστε να βελτιωθεί η απόδοση του μοντέλου.

#### f) Cross-Validation to find the optimal tree size.

Για να βρούμε το ιδανικό μέγεθος του δέντρου και να αποφύγουμε το φαινόμενο του overfitting, τρέχουμε την εξής συνάρτηση:

```
> cv_oj <- cv.tree(oj_tree, FUN=prune.misclass)
```

Η εντολή αυτή εκτελεί K-fold Cross-Validation στα δεδομένα εκπαίδευσης. Το [FUN = prune.misclass](#), δηλώνει στην R ότι θέλουμε το Cross-Validation να καθοδηγηθεί με βάση το πραγματικό ποσοστό σφάλματος ταξινόμησης και όχι με βάση το deviance.

Το cv\_oj που μας επιστρέφει περιλαμβάνει 3 βασικά διανύσματα:

- **\$size** : Το πλήθος των τερματικών κόμβων για κάθε δέντρο που εξετάστηκε.
- **\$dev** : Το συνολικό σφάλμα που αντιστοιχεί σε κάθε μέγεθος δέντρου κατά το Cross-Validation.
- **\$k** : Την παράμετρο κόστους πολυπλοκότητας που ελέγχει την ποινή για την προσθήκη επιπλέον κόμβων.

```

> cv_oj
$size
[1] 8 5 3 2 1

$dev
[1] 141 138 161 165 313

$k
[1] -Inf 0 8 11 154

$method
[1] "misclass"

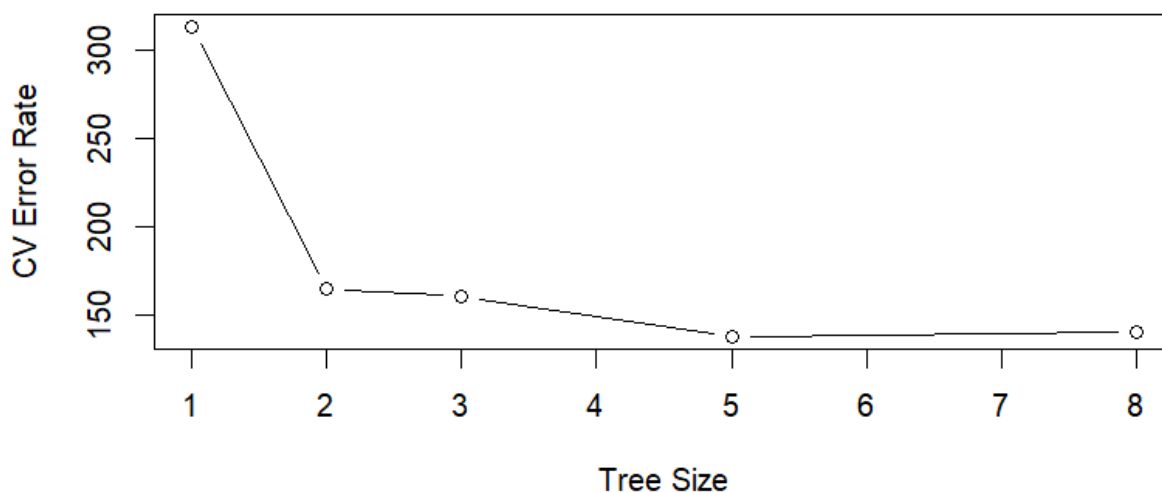
attr(,"class")
[1] "prune" "tree.sequence"

```

### g) Cross-Validation Plot

Με βάση τα αποτελέσματα του Cross-Validation, κατασκευάστηκε το ακόλουθο διάγραμμα που δείχνει τη συσχέτιση του μεγέθους του δέντρου (tree size) και του σφάλματος ταξινόμησης (CV error rate).

Παρατηρούμε ότι το γράφημα έχει μια τυπική καμπύλη με σχήμα V. Καθώς ξεκινάμε απλό το πιο απλό δέντρο με 1 κόμβο, όπου παρουσιάζεται το μεγαλύτερο σφάλμα 313 λαθών, παρατηρούμε ότι το σφάλμα μειώνεται καθώς το δέντρο αποκτά περισσότερους κόμβους. Το χαμηλότερο σημείο της καμπύλης είναι στους 5 κόμβους με 138 λάθη. Όταν το δέντρο γίνεται πιο σύνθετο και φτάνει στους 8 κόμβους, το σφάλμα αυξάνεται σε 141, πράγμα που επιβεβαιώνει την ύπαρξη overfitting στο δέντρο.



### h) Optimal Tree Size that corresponds to the lowest CV Error Rate.

Εξετάζοντας αναλυτικά την έξοδο του Cross-Validation, το βέλτιστο μέγεθος του δέντρου προκύπτει από την ελαχιστοποίηση του σφάλματος ταξινόμησης (τιμή \$dev).

Η εύρεση στην R γίνεται με τον εντοπισμό της θέσης όπου το σφάλμα γίνεται ελάχιστο (138). Αυτή θέση αντιστοιχεί σε ένα δέντρο με 5 τερματικούς κόμβους.

Συνεπώς, η στατιστική ανάλυση υποδεικνύει ότι το κλαδεμένο δέντρο με 5 κόμβους αποτελεί την ιδανική επιλογή, καθώς επιτυγχάνει τη βέλτιστη ισορροπία μεταξύ της απλότητας του μοντέλου και της ικανότητας του να γενικεύει σωστά τα δεδομένα, αποφεύγοντας όμως το overfitting.

```
> best_size <- cv_oj$size[which.min(cv_oj$dev)]
> cat("Optimal Tree Size:", best_size, "\n")
Optimal Tree Size: 5
```

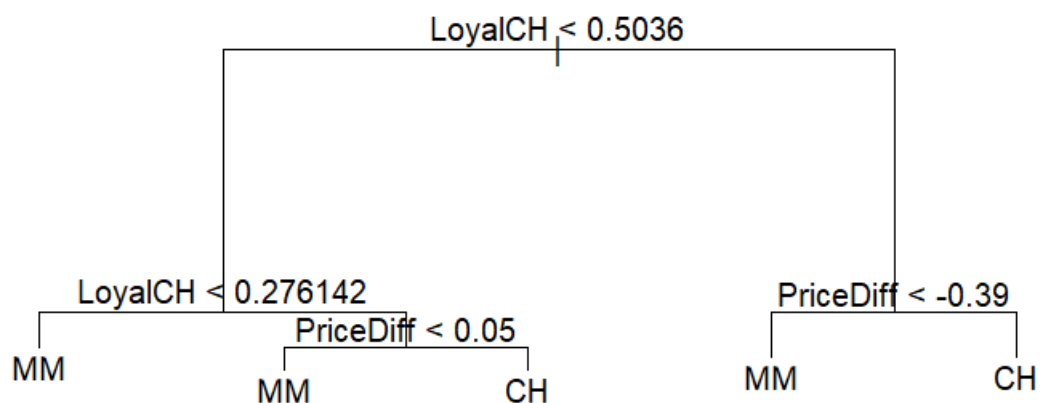
### i) Pruning the Tree

Αφού βρήκαμε το βέλτιστο μέγεθος, προχωράμε στο κλάδεμα του αρχικού δέντρου με σκοπό την απλοποίηση του.

Για να πραγματοποιήσουμε το κλάδεμα στην R χρησιμοποιήσαμε έναν δυναμικό έλεγχο με τις εντολές [if/else](#), οι οποίες στην περίπτωση που το Cross-Validation πρότεινε να μην γίνει κάποια αλλαγή, εξαναγκάζουν το μοντέλο να κλαδευτεί στους 5 κόμβους. Στην περίπτωση αυτή, το δέντρο κλαδεύεται επίσημα στους 5 τερματικούς κόμβους μέσω της συνάρτησης [prune.misclass\(\)](#).

Μέσω του κλαδέματος του δέντρου αφαιρούμε αναδρομικά τις χαμηλότερες διακλαδώσεις με την λιγότερη πληροφορία, αποκτώντας έτσι ένα πιο δυνατό και εύκολα ερμηνεύσιμο μοντέλο που πλέον δεν έχει τον θόρυβο των δεδομένων εκπαίδευσης.

```
> prune_size <- if(best_size == max(cv_oj$size)) 5 else best_size
> pruned_oj <- prune.misclass(oj_tree, best = prune_size)
> plot(pruned_oj)
> text(pruned_oj, pretty = 0)
```



#### j) Training error rates comparison between the pruned and unpruned trees.

Συγκρίνοντας τα Training Error Rates μεταξύ των unpruned και pruned δέντρων, παρατηρούμε ότι οι τιμές είναι ακριβώς ίδιες, με το σφάλμα να διαμορφώνεται στο 16,5%.

Αυτό οφείλεται στο γεγονός ότι οι κόμβοι που αφαιρέθηκαν κατά τη διαδικασία του κλαδέματος περιείχαν ελάχιστες παρατηρήσεις ή διαχωρισμούς που δεν επηρέασαν το τελικό συνολικό πλήθος των λανθασμένων ταξινομήσεων στο Training Set.

Ο αλγόριθμος κατάφερε να απλοποιήσει το δέντρο από τους 8 στους 5 κόμβους χωρίς να θυσιάσει την αρχική του ικανότητα προσαρμογής στα δεδομένα εκπαίδευσης.

```
> pruned_pred_train <- predict(pruned_oj, oj_train, type = "class")
> pruned_train_error <- mean(pruned_pred_train != oj_train$Purchase)
```

```
> cat("Unpruned Train Error:", train_error, "| Pruned Train Error:", pruned_train_error, "\n")
Unpruned Train Error: 0.165 | Pruned Train Error: 0.165
```

#### k) Test error rates comparison between the pruned and unpruned trees

Συγκρίνοντας τα Test Error Rates στα άγνωστα δεδομένα, βλέπουμε πως και σε αυτή την περίπτωση οι τιμές παραμένουν ακριβώς ίδιες με το σφάλμα να είναι στο 18,51% και για τα δύο δέντρα.

Συνήθως, το κλάδεμα μειώνει το σφάλμα ελέγχου προστατεύοντας το μοντέλο από το overfitting. Εδώ, η διατήρηση του ίδιου ακριβώς σφάλματος μας δείχνει πως οι κόμβοι που αφαιρέθηκαν δεν πρόσθεταν θόρυβο, αλλά ούτε και σημαντική πληροφορία για τις προβλέψεις του Test Set.

Το δέντρο με τους 5 κόμβους προσφέρει την ίδια ακρίβεια με το δέντρο των 8 κόμβων, είναι όμως πιο σύντομο, εύκολα κατανοητό και επομένως προτιμότερο.

```
> pruned_pred_test <- predict(pruned_oj, oj_test, type = "class")
> pruned_pred_error <- mean(pruned_pred_test != oj_test$Purchase)
```

```
> cat("Unpruned Test Error:", test_error, "| Pruned Test Error:", pruned_pred_error, "\n")
Unpruned Test Error: 0.1851852 | Pruned Test Error: 0.1851852
```

### 3. Άσκηση 3 – Δέντρα Παλινδρόμησης με το Car seats Dataset

#### a) Train/Test Split

Για τη σωστή αξιολόγηση της ικανότητας του μοντέλου, το αρχικό σύνολο δεδομένων Carseats που περιέχει 400 παρατηρήσεις και 11 στήλες, χωρίστηκε σε δύο ισοδύναμα μέρη με τη χρήση της συνάρτησης [sample\(\)](#).

**Σύνολο Εκπαίδευσης (Training Set):** Περιλαμβάνει ένα τυχαίο δείγμα με 200 παρατηρήσεις.

**Σύνολο Ελέγχου (Test Set):** Περιλαμβάνει τις υπόλοιπες 200 παρατηρήσεις που απέμειναν, οι οποίες θα χρησιμοποιηθούν για την αξιολόγηση της προβλεπτικής ικανότητας.

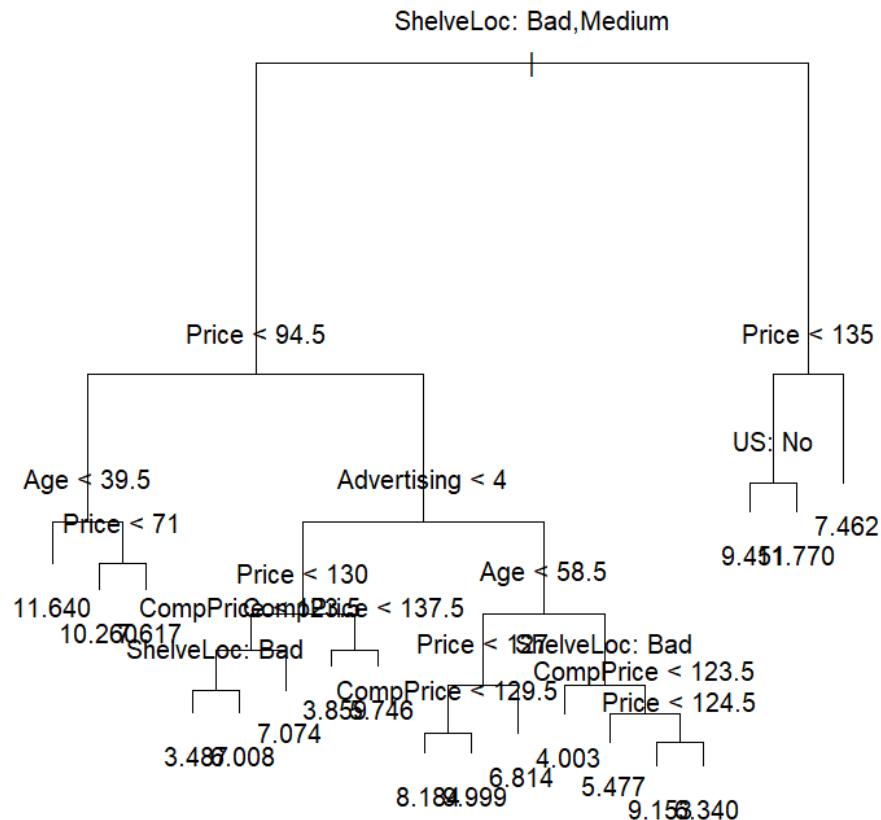
Η γεννήτρια τυχαίων αριθμών σταθεροποιήθηκε με την εντολή [set.seed\(1\)](#) για να διασφαλίσουμε την αναπαραγωγικότητα των αποτελεσμάτων.

```
> set.seed(1)
> train <- sample(1:nrow(Carseats), nrow(Carseats)/2)
> car_train <- Carseats[train, ]
> car_test <- Carseats[-train, ]
```

#### b) Fitting and Plotting the Regression Tree and Calculating MSE

Προσαρμόζουμε το Regression Tree με εξαρτημένη μεταβλητή της πωλήσεις. Συνεχίζουμε δημιουργώντας το γράφημα του δέντρου.

```
> car_tree <- tree(Sales~ ., data = car_train)
> plot(car_tree)
> text(car_tree, pretty = 0)
```



Η γραφική απεικόνιση του δέντρου μας δείχνει ότι ο πρώτος και ο πιο καθοριστικός διαχωρισμός (root node) πραγματοποιείται με βάση τη μεταβλητή **ShelveLOC**.

Αν η θέση του ραφίου είναι κακή ή μέτρια, το δέντρο μας οδηγεί στην αριστερή πλευρά, όπου εξετάζονται δευτερεύοντες παράγοντες όπως είναι η τιμή, η ηλικία και η διαφήμιση.

Αν η θέση είναι καλή, τότε μας οδηγεί στην δεξιά πλευρά, όπου οι αναμενόμενες πωλήσεις είναι υψηλότερες.

### Αξιολόγηση MSE:

Από την εκτέλεση του κώδικα προέκυψαν τα εξής αποτελέσματα για το MSE

**Train MSE:** 1,9716

**Test MSE:** 4,9220

```
> pred_train <- predict(car_tree, car_train)
> train_mse <- mean((pred_train - car_train$Sales)^2)
> pred_test <- predict(car_tree, car_test)
> test_mse <- mean((pred_test - car_test$Sales)^2)
> cat("Train MSE:", train_mse, "\n")
Train MSE: 1.971696
> cat("Test MSE:", test_mse, "\n")
Test MSE: 4.922039
```



Παρατηρώντας τα αποτελέσματα, καταλαβαίνουμε αμέσως ότι υπάρχει το φαινόμενο του overfitting στο μοντέλο μας. Το Test MSE με 4,92 είναι κατά πολύ μεγαλύτερο από το Train MSE που είναι 1,97. Το μοντέλο αναπτύχθηκε σε μεγάλο βάθος και κράτησε τον θόρυβο από τα δεδομένα εκπαίδευσης χάνοντας σε μεγάλο βαθμό την γενικευτική του ισχύ όταν προσπάθησε να προβλέψει νέες παρατηρήσεις από το Test Set.

### c) Cross-Validation and Pruning the Tree. Does it improve the test/train MSE?

Για να βρούμε το βέλτιστο μέγεθος του δέντρου και να αντιμετωπίσουμε το overfitting, εφαρμόζουμε Cross-Validation με την βοήθεια της συνάρτησης `cv.tree(car_tree)`.

```
> cv_car <- cv.tree(car_tree)
> print(cv_car)
$size
 [1] 18 17 16 15 14 13 12 11 10  8  7  6  5  4  3  2  1

$dev
 [1] 1127.545 1178.938 1189.224 1169.866 1178.081 1220.430 1237.090 1235.835 1235.83
 5 1256.516
[11] 1265.430 1207.840 1329.700 1373.358 1390.672 1405.433 1666.797

$k
 [1]      -Inf  16.99544  20.56322  25.01730  25.57104  28.01938  30.36962  31.56747
31.80816
[10]  40.75445  44.44673  52.57126  76.21881  99.59459 116.69889 159.79501 337.60153

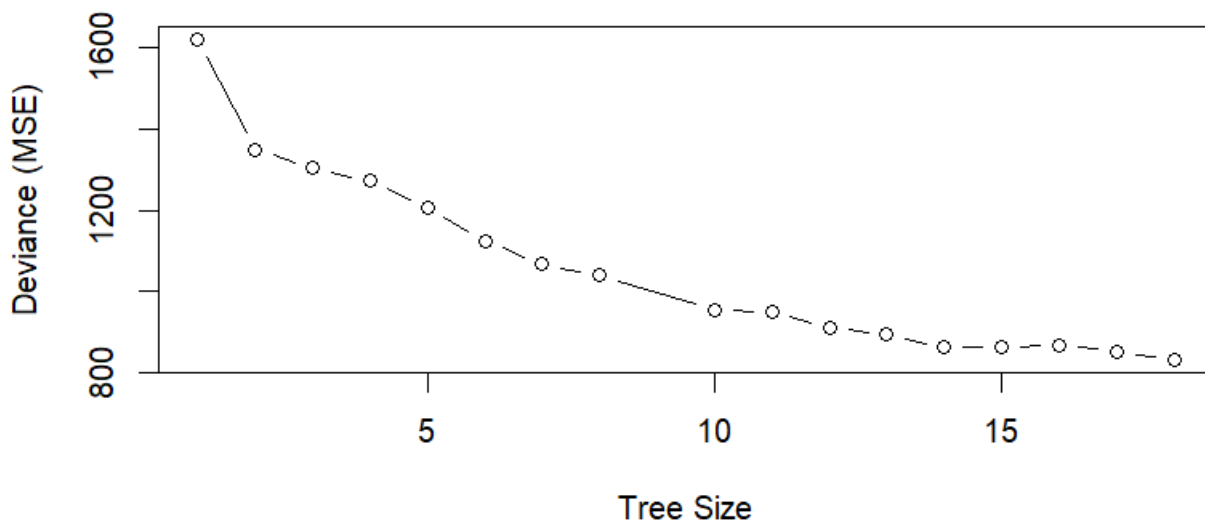
$method
[1] "deviance"

attr("class")
[1] "prune"      "tree.sequence"
> plot(cv_car$size, cv_car$dev, type = "b", xlab = "Tree Size", ylab = "Deviance (MSE)")
```

Το διάγραμμα που κατασκευάστηκε μας δείχνει την συσχέτιση ανάμεσα στο μέγεθος του δέντρου, ανάλογα με το πλήθος των κόμβων και το συνολικό σφάλμα.

Στην αριστερή πλευρά του γραφήματος, όταν το δέντρο έχει λίγους κόμβους, το σφάλμα είναι υψηλό ξεπερνά το 1600. Αυτό οφείλεται στο underfitting, δηλαδή το μοντέλο είναι υπερβολικά απλό για να αποτυπώσει τις σχέσεις των δεδομένων.

Καθώς αυξάνονται οι κόμβοι και κινούμαστε προς τα δεξιά, παρατηρούμε ότι η καμπύλη έχει μια συνεχή φθίνουσα πορεία. Η καμπύλη βρίσκεται στο ελάχιστον σημείο της στους 18 τερματικούς κόμβους, όπου και εμφανίζεται και το χαμηλότερο συνολικό σφάλμα.



Επιβεβαιώσαμε το βέλτιστο μέγεθος του δέντρου μέσα από την εντολή που βλέπουμε κάτω. Η συνάρτηση `which.min()`, εντόπισε τη θέση του ελάχιστου σφάλματος επιστρέφοντας ως βέλτιστο μέγεθος τους 18 κόμβους.

Καθώς το προτεινόμενο μέγεθος του δέντρου συμπίπτει με το μέγεθος του αρχικού δέντρου, η συνάρτηση [`prune.tree\(\)`](#) επιστρέφει ένα μοντέλο που είναι ίδιο με το αρχικό.

Μέσω της συνάρτησης `summary(pruned_car)`, η R μας επιβεβαιώνει ότι το δέντρο διατηρεί τους 18 τερματικούς κόμβους μετά το κλάδεμα.

```
> best_car_size <- cv_car$size[which.min(cv_car$dev)]
> cat("Optimal Tree Size:", best_car_size, "\n")
Optimal Tree Size: 18

> pruned_car <- prune.tree(car_tree, best = best_car_size)
> summary(pruned_car)
```

Regression tree:  
`tree(formula = Sales ~ ., data = car_train)`  
 Variables actually used in tree construction:  
 [1] "ShelveLoc" "Price" "Age" "Advertising" "CompPrice" "US"  
 Number of terminal nodes: 18  
 Residual mean deviance: 2.167 = 394.3 / 182  
 Distribution of residuals:

| Min.     | 1st Qu.  | Median   | Mean    | 3rd Qu. | Max.    |
|----------|----------|----------|---------|---------|---------|
| -3.88200 | -0.88200 | -0.08712 | 0.00000 | 0.89590 | 4.09900 |

Υπολογίζοντας από την αρχή τα MSE για το κλαδεμένο δέντρο, προκύπτουν τα ακόλουθα αποτελέσματα:

**Pruned Train MSE: 1,9716**

**Pruned Test MSE: 4,9220**

```
> pruned_train_mse <- mean((predict(pruned_car, car_train) - car_train$Sales)^2)
> pruned_test_mse <- mean((predict(pruned_car, car_test) - car_test$Sales)^2)
> cat("Pruned Tree - Train MSE:", pruned_train_mse, "|Test MSE:", pruned_test_mse, "\n")
Pruned Tree - Train MSE: 1.971696 |Test MSE: 4.922039
```

Εξετάζοντας τα νούμερα, παρατηρούμε ότι το κλάδεμα δεν βελτίωσε, ούτε το Test MSE, ούτε το Train MSE. Οι τιμές παρέμειναν οι ίδιες με το αρχικό δέντρο, καθώς μέσα από το Cross-Validation καταλάβαμε ότι κάθε ένας από τους 18 τερματικούς κόμβους προσφέρει χρήσιμη πληροφορία για την μείωση του deviance στο Training Set.

Η αδυναμία του κλαδέματος να μειώσει το Test MSE σημαίνει ότι η απλή δομή ενός δέντρου απόφασης έχει φτάσει στα όρια της απόδοσης της. Το μοντέλο συνεχίζει να έχει υψηλή διασπορά (variance), γεγονός που καθιστά αναγκαία την μετάβαση σε συνδυαστικές μεθόδους όπως το Bagging και τα Random Forests.

#### d) Bagging approach to analyze the data and importance of variables.

Το Bagging χρησιμοποιείται για να αντιμετωπίσει το overfitting και την υψηλή διασπορά του μεμονωμένου δέντρου απόφασης. Η μέθοδος λειτουργεί δημιουργώντας πολλαπλά υποδείγματα των δεδομένων εκπαίδευσης μέσω επαναεισαγωγής (bootstrapping), κατασκευάζοντας έτσι ένα πλήρες δέντρο για κάθε υπόδειγμα και συνδυάζοντας τις προβλέψεις τους.

Χρησιμοποιήσαμε την συνάρτηση `randomForest()` με την παράμετρο `mtry = 10`, καθώς το Car seat dataset έχει 10 διερμηνευτικές μεταβλητές. Το `mtry = 10` αναγκάζει τον αλγόριθμο να εξετάσει όλες τις διαθέσιμες μεταβλητές σε κάθε διακλάδωση.

Από την εκτέλεση του μοντέλου προέκυψαν οι ακόλουθες τιμές για το MSE:

**Bagging – Train MSE: 0.5014**

**Bagging – Test MSE: 2.6325**

Η βελτίωση σε σύγκριση με το Pruned Tree είναι εμφανής. Το Train MSE μειώθηκε σε 0,5014 ενώ το Test MSE μειώθηκε σε 2,63. Αυτή η τεράστια μείωση δείχνει την ισχύ της μεθόδου Bagging.

Συνδυάζοντας τις προβλέψεις από εκατοντάδες δέντρα, ο αλγόριθμος κατάφερε να εξομαλύνει τις ακραίες προβλέψεις και να περιορίσει την διασπορά (variance).

```
> bag_car <- randomForest(Sales~ ., data = car_train, mtry = 10, importance = TRUE)
> bag_train_mse <- mean((predict(bag_car, car_train) - car_train$Sales)^2)
> bag_test_mse <- mean((predict(bag_car, car_test) - car_test$Sales)^2)
> cat("Bagging - Train MSE:", bag_train_mse, "|Test MSE:", bag_test_mse, "\n")
Bagging - Train MSE: 0.501427 |Test MSE: 2.631581
```

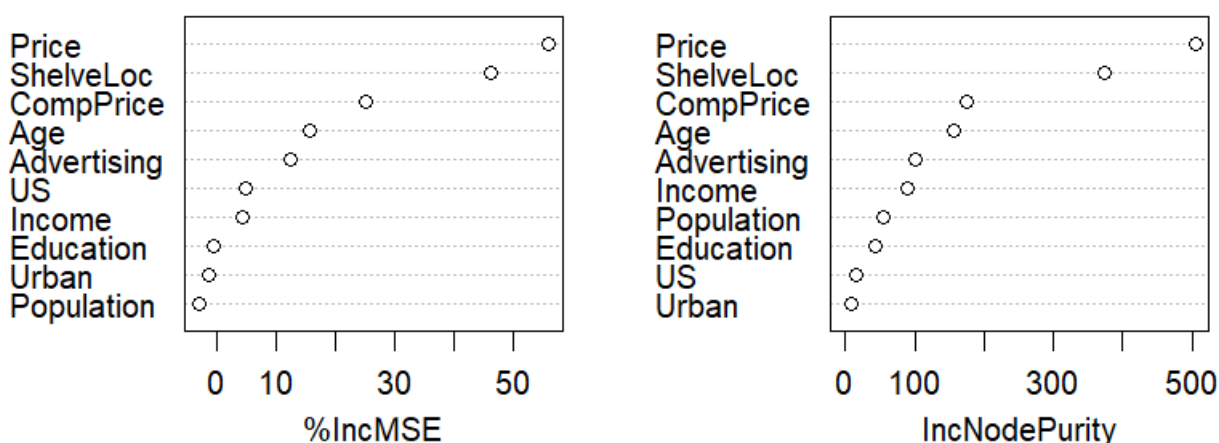
Για να προσδιορίσουμε ποιες μεταβλητές είναι σημαντικότερες και επηρεάζουν περισσότερο τις πωλήσεις, χρησιμοποιήσαμε την συνάρτηση `importance(bag_car)` και για να πάρουμε την γραφική της απεικόνιση την `varImpPlot()`.

Με βάση τους δείκτες **%IncMSE**, που μετράει την αύξηση του σφάλματος του μοντέλου, αν αφαιρεθεί η μεταβλητή, και του **IncNodePurity**, που μετράει την συνολική μείωση του deviance στους κόμβους λόγω τις μεταβλητής, προκύπτουν τα εξής:

- **Price**: Αποτελεί τον πιο κρίσιμο οικονομικό παράγοντα. Η τιμή του προϊόντος επηρεάζει άμεσα τη ζήτηση.
- **ShelveLoc**: Είναι εξίσου κυρίαρχη μεταβλητή και επιβεβαιώνει ότι το σημείο τοποθέτησης του καθίσματος στο κατάστημα επηρεάζει σε μεγάλο βαθμό την αγορά του προϊόντος από τους καταναλωτές.

```
> importance(bag_car)
              %IncMSE  IncNodePurity
CompPrice    25.0952148    175.901941
Income        4.4807016     90.391926
Advertising   12.4117147    101.577150
Population    -2.9573186     54.045384
Price         55.9745688    504.806041
ShelveLoc     46.1596900    372.698485
Age           15.6844766    157.157234
Education     -0.4736739     44.445906
Urban         -1.1516758      8.334879
US             4.8236824     16.768571
> varImpPlot(bag_car, main = "Bagging Variable Importance")
```

### Bagging Variable Importance



#### e) Random Forests analyze the data and variable importance.

Τα Random Forests αποτελούν μια εξέλιξη του Bagging και έχουν στόχο την αποσυσχέτιση των παραγόμενων δέντρων. Όταν δημιουργούνται πολλά δέντρα και υπάρχει μια ισχυρή μεταβλητή, όπως στο παράδειγμα μας είναι η **Price**, όλα τα δέντρα την χρησιμοποιούν στον πρώτο κόμβο, με αποτέλεσμα να μοιάζουν μεταξύ τους και να παρουσιάζουν έντονη συσχέτιση.

Τα Random Forests επιλύουν αυτό το πρόβλημα εισάγοντας την τυχαιότητα. Σε κάθε διακλάδωση του δέντρου επιτρέπεται η χρήση μόνο ενός τυχαίου υποσυνόλου  $m$  μεταβλητών. Στην περίπτωση μας ορίσαμε **mtry = 3**, δηλαδή πήραμε 3 από τις 10 μεταβλητές.

```
> rf_car <- randomForest(Sales~ ., data = car_train, mtry = 3, importance = TRUE)
> rf_train_mse <- mean((predict(rf_car, car_train) - car_train$Sales)^2)
> rf_test_mse <- mean((predict(rf_car, car_test) - car_test$Sales)^2)
> cat("Random Forest - Train MSE:", rf_train_mse, "| Test MSE:", rf_test_mse, "\n")
Random Forest - Train MSE: 0.6861678 | Test MSE: 3.070883
```

Από την εκτέλεση του μοντέλου προέκυψαν οι εξής τιμές:

- **Random Forest – Train MSE:** 0,6861
- **Random Forest – Test MSE:** 3,07

Η παράμετρος  $m$ , δηλαδή ο αριθμός των μεταβλητών που εξετάζονται σε κάθε split, παίζει καθοριστικό ρόλο στην απόδοση του μοντέλου.

- **Όταν το  $m$  είναι μεγάλο π.χ.  $m=10$  όπως στο Bagging**, ο αλγόριθμος έχει στη διάθεση του όλες τις μεταβλητές, οπότε πετυχαίνει πολύ χαμηλό σφάλμα εκπαίδευσης. Τα δέντρα όμως συνεχίζουν να συσχετίζονται έντονα μεταξύ τους, περιορίζοντας την μείωση της διασποράς.
- **Όταν το  $m$  μειώνεται, π.χ.  $m=3$  όπως στο Random Forest**, ο αλγόριθμος αναγκάζεται να προβλέψει ανάμεσα σε 3 τυχαίες μεταβλητές και αυτό έχει ως αποτέλεσμα να μειώνεται η συσχέτιση μεταξύ των δέντρων. Κάθε δέντρο αναγκάζεται να εξετάσει και άλλες μεταβλητές. Αυτό αυξάνει συνήθως το Train MSE, αλλά προσφέρει γενικά ένα πιο σταθερό μοντέλο.

Στην συγκεκριμένη ανάλυση, το Bagging  $m=10$  πέτυχε καλύτερο Test MSE (2.63) από το Random Forest  $m=3$  με Test MSE= 3.7. Αυτό μας δείχνει ότι οι κυρίαρχες μεταβλητές είναι κρίσιμες και ο περιορισμός τους σε 3 στέρησε μέρος της προβλεπτικής ικανότητας των δέντρων.

```
> importance(rf_car)
      %IncMSE IncNodePurity
CompPrice    15.3620526    156.79771
Income        3.1692502    120.90880
Advertising   7.8046269    108.57608
Population   -2.5512335    100.56722
Price        36.5947531    396.36763
ShelveLoc    35.9522295    290.53078
Age          11.4111604    174.34911
Education     0.4647345     72.66570
Urban         1.4222554     15.61579
US            6.3063458     34.71705
> varImpPlot(rf_car, main="Random Forest Variable Importance")
```

Με βάση τα αποτελέσματα που πήραμε από την συνάρτηση [importance\(\)](#) και τους δείκτες **%IncMSE** και **IncNodePurity**, οι μεταβλητές με την μεγαλύτερη επίδραση είναι:

- **Price** : Η τιμή παραμένει ο ισχυρότερος οικονομικός παράγοντας. Αν αποκλειστεί από το μοντέλο, το σφάλμα αυξάνεται δραματικά.
- **ShelveLoc**: Η θέση του προϊόντος στα ράφια ακόλουθεί την τιμή με μικρή διαφορά, επιβεβαιώνοντας την σημαντικότητα της τοποθέτησης στα καταστήματα και του μαρκετινγκ.
- **CompPrice**: Η τιμή του ανταγωνισμού αναδεικνύεται ως η τρίτη σημαντική μεταβλητή και μας δείχνει ότι οι καταναλωτές λειτουργούν συγκριτικά.

### Random Forest Variable Importance

